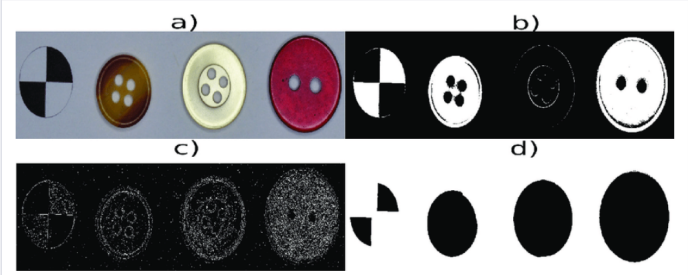
Discussion 6: Thresholding vs Segmentation

Discussion Topic:

Consider this image from Researchgate.



Applications of segmentation techniques by E. M. Sandoval, 2017.
https://doi.org/10.5772/intechopen.71539. CC BY 3.0

Applications of segmentation techniques.
Applications of segmentation techniques by E. M. Sandoval, 2017.
https://doi.org/10.5772/intechopen.71539. CC BY 3.0

Which do you think is the result of thresholding, and which is the result of region-based segmentation? Give detailed explanations for your choices.

Then, for each technique (thresholding and region-based), discuss one real world problem where that method would be optimal over the other for finding a solution. Give detailed support for your claims by discussing criteria such as the following:

Computational cost
Noise sensitivity
Accuracy of results
Speed of implementation
Flexibility of choice between interactive and automatic techniques

My Post:

Hello Class,

Looking at the images, I would say that image (b) is the result of thresholding and image (d) is the result of region-based segmentation. Image (a) is the original image, and image (c) seems to be the result of edge-based segmentation. The original image shows several clothing buttons on a light background. What I notice when comparing the processed images is that both techniques separate the objects from the background, but they do this differently. Image (b) appears to use the differences in pixel intensity to separate the buttons from the background, while image (d) uses a method that considers the neighboring pixels to identify regions. The images are from Martínez Sandoval et al. (2017), who compared these segmentation techniques using MATLAB.

To understand the difference between the two methods, I think we need to look at how they process the pixels. Thresholding is one of the simpler segmentation methods. For example, if we use OpenCV's `cv2.threshold()` function with `THRESH_BINARY`, the function compares each pixel's grayscale intensity against a selected threshold value. If the pixel value is greater than the threshold, it is assigned 255 (white); otherwise, it is assigned 0 (black). This produces a binary image containing only two intensity values (OpenCV, n.d.-b).

There are two approaches to thresholding that I think are worth mentioning, global and local thresholding. Global thresholding uses the same threshold value for the entire image. Local thresholding, also called adaptive thresholding, works a little differently because the threshold value is calculated using the neighboring pixels. OpenCV provides `cv2.adaptiveThreshold()`, which supports `ADAPTIVE_THRESH_MEAN_C` and `ADAPTIVE_THRESH_GAUSSIAN_C`. The first method calculates a local mean intensity, while the second uses a Gaussian-weighted mean. Both methods subtract a constant from the calculated value to determine the threshold (OpenCV, n.d.-b).

I have to say, at first I was thinking that local thresholding and region-based segmentation were closely related, because both methods examine neighboring pixels. However, there is a difference. Local thresholding uses neighboring pixels to calculate the threshold value for each pixel. Region-based segmentation uses neighboring pixels to determine whether they should be grouped into the same region. In other words, one is calculating a threshold, while the other is trying to identify connected regions that share similar characteristics.

This is why I would identify image (d) as region-based segmentation. The module (Colorado State University Global, n.d.) explains that region-based segmentation examines characteristics such as intensity, color, and texture to group pixels into regions. There are three methods discussed in the module: region growing, region splitting and merging, and watershed.

For example, region growing starts with one or more selected pixels, called seeds. From those pixels, the algorithm examines their neighbors and determines whether they have similar characteristics. If they meet the selected criteria, they are added to the region. The process continues by examining neighboring pixels until no additional pixels meet the conditions. What I find interesting about this method is that the algorithm is not simply deciding whether a pixel should be black or white; it is also examining its relationship with the pixels around it. Note that a region-based segmentation method can also produce a binary image. So, the fact that images (b) and (d) are black and white does not necessarily tell us which algorithm was used. We have to consider how the regions were constructed. Also, the figure does not tell us exactly which region-based algorithm produced image (d), only that it is the region-based result.

For a real-world application of thresholding, I would choose document scanning, more especially when preparing documents for Optical Character Recognition (OCR). In my Module 5 Critical Thinking Assignment, I worked with a scanned handwritten note that was attached to a printed form. After extracting the note, I used Otsu's thresholding to convert the image into a binary image before applying morphological operations.

I think thresholding makes sense for this type of application because what we need first is to separate the writing from the paper. If the background is reasonably uniform and there is enough contrast between the writing and the paper, a global threshold can accomplish this without much processing.

Otsu's method is also useful because it calculates a threshold automatically using the image's intensity histogram. This means we do not have to manually select a threshold value for each image (OpenCV, n.d.-b).

For example, if a company has thousands of scanned documents to process, I would probably start with thresholding rather than a more complicated region-based method. The computational cost is relatively low, and implementing the operation using OpenCV is fairly simple. Processing each pixel using a fixed threshold also requires fewer calculations than examining neighborhoods or constructing regions.

However, noise and lighting can cause problems. If the paper contains small marks, those marks might become part of the foreground. The opposite can also happen with faint handwriting, where portions of a letter disappear because their intensity is too close to the background. I would consider applying Gaussian smoothing before thresholding to reduce small intensity variations caused by noise. If the image has shadows or uneven lighting, I would probably use adaptive thresholding instead.

Nonetheless, adaptive thresholding requires additional calculations, and smoothing might remove some of the fine details from the handwriting. There is a tradeoff here. A cleaner binary image is useful, but if we remove part of a letter, that information might be needed later by the OCR system. A good thresholding result does not necessarily mean that the handwriting will be recognized correctly.

For region-based segmentation, I would choose a different application, such as an industrial inspection system that needs to identify and count mechanical parts. Suppose a camera takes a picture of several washers on a conveyor belt. If the washers are separated, thresholding might be enough to distinguish them from the background. However, if two washers are touching, thresholding might classify them as one object, and if the system is supposed to count individual washers, that would produce an incorrect result. For this particular example, marker-based watershed segmentation may be better. OpenCV has an example using several touching coins, which is very similar to the washers example. First, thresholding is applied to obtain an initial binary image. The opening operation, a morphological operation, is then used to remove some of the noise, and a distance transform helps identify interior areas of the objects. From there, the program creates markers for the regions and uses cv2.watershed() to determine the boundaries between them (OpenCV, n.d.-a).

The watershed method is more complicated than basic thresholding, and it requires additional processing to identify regions and create markers. These markers can be selected manually or generated automatically by the program. In the OpenCV example, the markers are generated automatically using cv2.connectedComponents(). We need to be careful with these markers. If the algorithm creates too many incorrect markers, it might divide one washer into several regions. If the markers do not properly identify the individual objects, two touching washers might remain grouped together. The OpenCV example actually shows that some touching coins are separated correctly while others are not (OpenCV, n.d.-a). So, region-based segmentation does not automatically guarantee a better result. Still, for this application, I think the additional computational cost could be justified. What matters is whether the system can identify the individual parts correctly. A faster method would not be very useful if the system consistently counted two washers as one.

For automatic and interactive processing, I would prefer automatic methods for both applications. With thresholding, we can select a value manually, use Otsu's method to calculate a global threshold, or use adaptive thresholding to calculate local values. For region-based segmentation, we can select the seeds

or markers manually, or have the program calculate them. I think interactive segmentation could be useful while developing and testing the program, especially when trying to understand why certain regions are not being identified correctly. However, manually selecting markers for every image would not be practical in an industrial system processing many parts.

Something else worth mentioning is that these techniques do not necessarily have to be used separately. In the OpenCV watershed example, thresholding is used as part of the preprocessing before the region-based segmentation is performed. So, one method can help prepare the image for the other.

The study by Martínez Sandoval et al. (2017) reported that thresholding was the fastest of the three segmentation techniques they tested, followed by edge-based and region-based segmentation. This makes sense to me because simple thresholding does not have to examine the relationships between neighboring pixels to construct regions. However, I would not choose a method based only on its processing speed. If I only need to separate writing from the background, thresholding would be my choice. But if I need to separate and count individual objects that are touching, I would prefer region-based segmentation, even if it requires more processing.

-Alex

References:

Colorado State University Global. (n.d.). *Lecture 6: Segmentation* [Course lecture]. Canvas.

Martínez Sandoval, E., Martínez Rosas, M. E., Martínez Sandoval, J. R., Miranda Velasco, M. M., & Cervantes de Ávila, H. (2017). Machine vision systems: A tool for automatic color analysis in agriculture. In S. Hussmann (Ed.), Automation in agriculture: Securing food supplies for future generations (pp. 125–148). IntechOpen. https://doi.org/10.5772/intechopen.71935

OpenCV. (n.d.-a). *Image segmentation with watershed algorithm*. https://docs.opencv.org/4.13.0/d3/db4/tutorial_py_watershed.html

OpenCV. (n.d.-b*). Image thresholding*. https://docs.opencv.org/4.13.0/d7/d4d/tutorial_py_thresholding.html